

API-driven Cloud Native Solutions

Assignment II — SupportSense Project Report

Group Details - Group No: 11

Repo : <https://huggingface.co/gowzrish/supportsense-distilbert-lora/tree/main>

<https://github.com/gowrii23/api-driven/tree/main/supportsense>

Sl. No	BITS ID	Name	Contribution of team member (Qualitative)	Percentage Contribution (Out of 100) Quantitative
1	2024AC05203	Kodhandan S	Data layer: FAQ corpus, sample complaints CSV, Banking77 category mapping for fine-tuning	100%
2	2024AC05046	Gowrishankar S	Fine-tuning: DistilBERT+LoRA training, HF Hub upload, fine-tuned classifier service	100%
3	2024AC05045	Arunkumar K A	API integration: Hugging Face Inference API services, orchestrate() pipeline, Gradio UI	100%
4	2024AC05219	Naveen R	Documentation: Word report, application screenshots, demo and viva preparation	100%
5	2024AC05304	KUMARESH BABU A	Testing and deployment: pytest smoke tests, environment setup, README and run scripts	100%

Abstract

SupportSense is an interactive, multi-modal Artificial Intelligence (AI) application developed for the customer-service domain as part of the API-driven Cloud Native Solutions. The system automates support-ticket triage by combining Natural Language Processing (NLP) and Computer Vision (CV) sub-tasks through a unified orchestration pipeline. The solution is explicitly API-driven: the majority of NLP capabilities are invoked through the Hugging Face Inference API, while a domain-adapted classifier, fine-tuned using Low-Rank Adaptation (LoRA) on the PolyAI Banking77 dataset, is published to and retrieved from the Hugging Face Model Hub at runtime. The implementation follows LLMops (Large Language Model Operations) and cloud-native operational principles, including structured logging with correlation identifiers, per-call metrics capture, prompt versioning, environment-based configuration, input guardrails, and graceful fallback paths, together with an offline evaluation harness that measures six operational and quality metrics. The interactive front end is built using Gradio and exposes a triage workflow, a fine-tuned-versus-zero-shot comparison view, and a live operational metrics dashboard, thereby satisfying the demonstrability requirement of the assignment.

1. Project Overview

SupportSense is an interactive customer-service AI assistant that triages support tickets by combining Natural Language Processing and Computer Vision. Agents enter a complaint (and optional product photo and FAQ question); the system classifies the issue, analyses sentiment, summarises the text, answers FAQ questions, checks image defects, captions images, and drafts an empathetic reply.

GitHub repository: <https://github.com/gowrii23/api-driven.git>

Application URL (local demo): <http://127.0.0.1:7860>

1.1 Problem Statement

Customer-support organisations routinely receive heterogeneous, unstructured inputs, including free-text complaints, policy-related questions, and, in some cases, product images. Manually triaging this volume of input is time-consuming, inconsistent across agents, and does not scale efficiently. SupportSense addresses this problem by orchestrating seven distinct AI sub-tasks into a single, cohesive pipeline:

- Classifies each complaint into an operational category.
- Analyses sentiment to enable tone-aware prioritisation.
- Summarises long narratives for faster agent readability.
- Answers policy or FAQ questions extractively from a knowledge base.
- Assesses attached product images for visible defects.
- Generates image captions that enrich the textual context.
- Drafts an empathetic, policy-grounded reply for agent review.

All seven sub-tasks converge in a single `orchestrate()` function, which returns a structured triage packet consumed by the Gradio interface, the logging subsystem, and the metrics subsystem, thereby satisfying the assignment's requirement that all sub-tasks be cohesive and work towards a unified objective.

2. Assignment Requirements Mapping

The table below maps each of the eight deliverables specified in the Assignment II problem statement to its corresponding implementation within SupportSense, demonstrating that every explicit requirement of the assignment brief has been satisfied.

- Domain: Customer Service — automated triage for support agents.
- Two AI categories: Natural Language Processing (6 sub-tasks) + Computer Vision (2 sub-tasks).
- Five+ sub-tasks: Seven sub-tasks: classification, sentiment, summarization, QA, reply generation, image defect check, image captioning.
- Models (LLM/SLM): DistilBERT+LoRA (fine-tuned), RoBERTa sentiment, DistilBART, RoBERTa-SQuAD, Zephyr-7B, CLIP, BLIP.
- Unified objective: All sub-tasks feed one `orchestrate()` pipeline producing a single triage packet for agents.
- APIs & interactive demo: Hugging Face Hub + Inference API; Gradio UI with Triage, Fine-tuned Demo, and Metrics tabs.
- LLMOps & metrics: Pinned models in `config.py`, prompt versioning, `.env` secrets, JSONL metrics, structured logs, evaluation harness (5+ metrics).
- Fine-tuning: DistilBERT + LoRA on PolyAI/banking77 (collapsed to 5 categories); model on HF Hub.

Taken together, requirements 1–3 establish the domain and scope of the project, requirement 4 identifies the AI models selected to fulfil that scope, requirement 5 establishes the architectural cohesion between sub-tasks, requirement 6 satisfies the API-driven and interactive-demonstration criteria, requirement 7 satisfies the LLMOps and metrics criterion, and requirement 8 satisfies the fine-tuning criterion. Sections 3 to 6 below elaborate on each of these in turn.

3. Sub-tasks and Models

Sub-task	Category	Model	Runtime
Complaint classification (fine-tuned)	NLP	gowzrish/supportsense-distilbert-classifier	HF Hub (local inference)
Sentiment analysis	NLP	cardiffnlp/twitter-roberta-base-sentiment-latest	HF Inference API
Summarization	NLP	sshleifer/distilbart-cnn-12-6	HF Inference API
FAQ question answering	NLP	deepset/roberta-base-squad2	HF Inference API

Reply generation	NLP	HuggingFaceH4/zephyr-7b-beta	HF Inference API
Image defect check	CV	openai/clip-vit-base-patch32	Local (transformers)
Image captioning	CV	Salesforce/blip-image-captioning-base	Local (transformers)

3.1 Inference Modes: Remote API vs. Local Execution

SupportSense deliberately mixes two inference modes, illustrating the trade-offs typically encountered in production AI systems. Sentiment analysis, summarisation, extractive question answering, and reply generation are invoked remotely through the Hugging Face Inference API using a single, centrally configured InferenceClient. This mode requires no local GPU, benefits from model updates and scaling managed by the provider, and is billed on a pay-per-use basis, at the cost of network latency, rate limiting, and dependency on third-party availability. In contrast, the fine-tuned classifier, the CLIP-based defect detector, and the BLIP image-captioning model are executed locally via the transformers library after being downloaded once from the Hugging Face Hub or cache. Local execution removes network latency and external dependency at inference time but requires local compute resources and does not benefit automatically from upstream model updates. This dual-mode design is intentional: custom fine-tuned classifiers are typically not available on the free serverless Inference API, so Hub-hosted weights combined with local inference is the standard pattern for assignment-scale custom models, whereas well-established, general-purpose models are more efficiently consumed as a managed remote service.

3.2 API-Driven, Cloud-Native Architecture

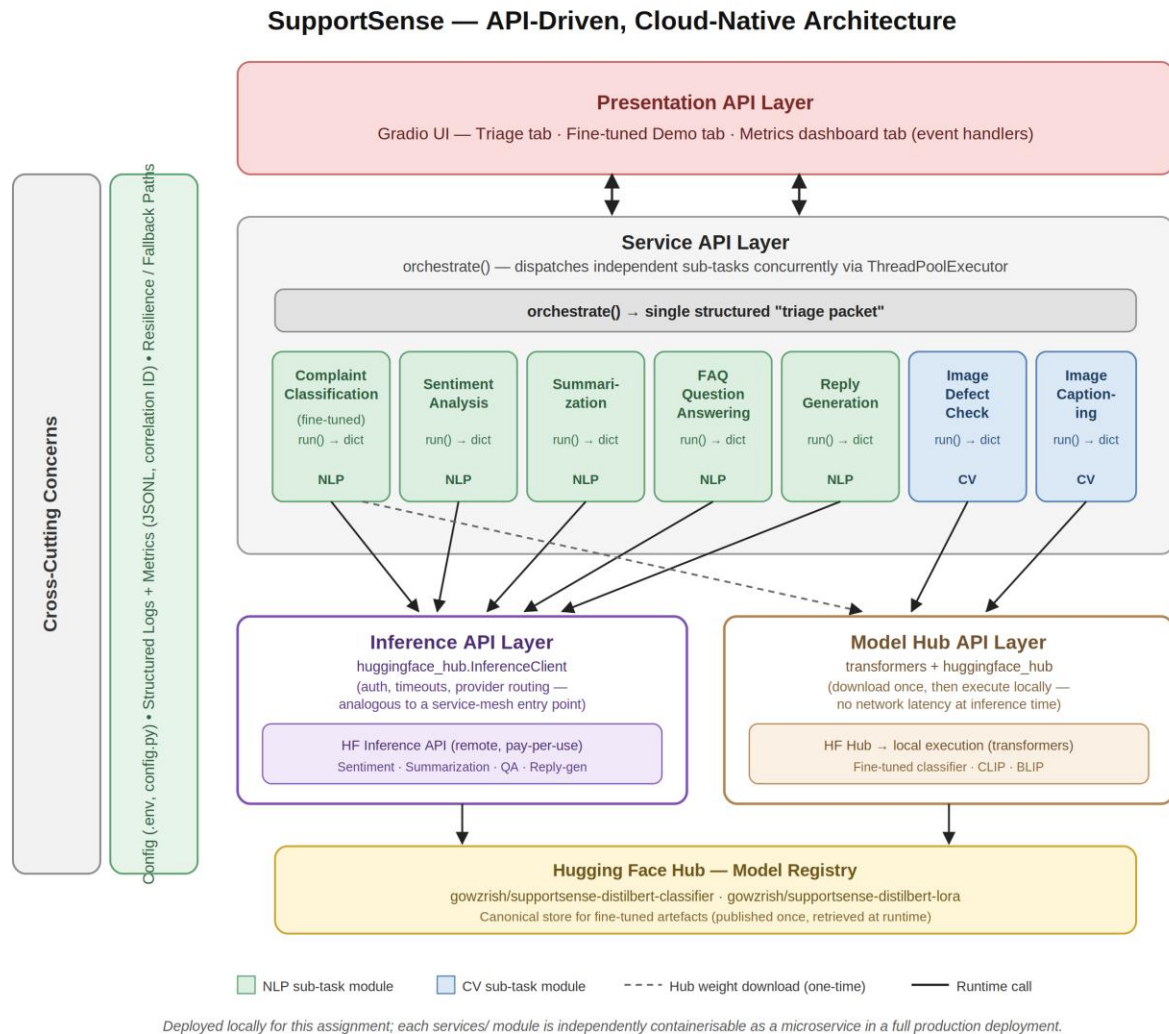


Figure 1: API-Driven, Cloud-Native Architecture of SupportSense, showing the Presentation, Service, Inference API, and Model Hub API layers together with the Hugging Face Hub model registry and cross-cutting operational concerns.

In cloud-native system design, API-driven architecture decouples application logic from model execution by invoking capabilities through well-defined remote interfaces rather than embedding monolithic models directly in the application binary. SupportSense reflects this pattern at four layers:

- Inference API layer (`huggingface_hub.InferenceClient`) — remote inference for sentiment, summarisation, question answering, and reply generation.
- Model Hub API layer (`transformers` + `huggingface_hub`) — downloads and runs the fine-tuned classifier at `gowzrish/supportsense-distilbert-classifier`.
- Internal Service API layer — each sub-task module exposes a uniform `run(...) → dict` contract.

- Presentation API layer — Gradio event handlers expose the pipeline interactively.

The central InferenceClient encapsulates authentication, timeouts, and provider routing in a manner analogous to a service-mesh entry point for external AI capabilities. Although the application is deployed locally for this assignment, its design aligns with cloud-native principles: decoupled services, externalised configuration, stateless orchestration, concurrent execution via ThreadPoolExecutor, observability through structured JSON logs and per-call metrics, resilience via fallback paths, and use of the Hugging Face Hub as a model registry. In production, each service module could be containerised and invoked over REST or a message queue.

4. Fine-Tuning Details

- Dataset: PolyAI/banking77 (13,083 banking customer-service utterances) mapped to five SupportSense categories: billing, delivery, product-defect, account, other.
- Base model: distilbert-base-uncased with LoRA (r=8, alpha=16, 3 epochs, macro-F1 selection).

Hugging Face Hub models:

- Merged classifier: <https://huggingface.co/gowzrish/supportsense-distilbert-classifier>
- LoRA adapter: <https://huggingface.co/gowzrish/supportsense-distilbert-lora>

4.1 Methodology and Rationale

Fine-tuning was performed using Low-Rank Adaptation (LoRA), a Parameter-Efficient Fine-Tuning (PEFT) technique in which the original weight matrices of a pre-trained model are frozen and a pair of small, trainable low-rank matrices is injected into selected layers to approximate the weight update required for the downstream task. The base model, distilbert-base-uncased, was selected as a Small Language Model (SLM) backbone: DistilBERT retains approximately 97% of BERT's language-understanding performance while using roughly 40% fewer parameters, which makes it well suited to a classification task with a modest label set and to training under limited compute resources. LoRA was applied to the query and value projection matrices (q_{lin} , v_{lin}) of the attention mechanism with rank $r = 8$, scaling factor $\alpha = 16$, and dropout 0.05, resulting in approximately 1% of the base model's parameters being trainable. This substantially reduces training time, memory footprint, and storage cost relative to full fine-tuning, while empirically retaining most of the accuracy gain, which is why PEFT methods such as LoRA are widely used to adapt SLMs and LLMs to narrow, domain-specific tasks. The PolyAI/banking77 dataset (13,083 labelled banking customer-service utterances spanning 77 fine-grained intents) was selected because banking-style support queries are closely aligned with SupportSense's customer-service triage domain; the 77 original intents were collapsed to the five operational categories used by the application (billing, delivery, product-defect, account, other) via keyword-based heuristics, so that the fine-tuned classifier's output vocabulary matches the categories consumed by the downstream reply-generation stage. Training was run for 3 epochs with a batch size of 32 (training) and 64 (evaluation), a learning rate of $2e-4$, and a maximum sequence length of 128 tokens, with the checkpoint selected by macro-F1

score to guard against class imbalance across the five categories. Two artefacts were subsequently published to the Hugging Face Hub: the LoRA adapter alone, retained for reproducibility and to illustrate the parameter-efficient delta explicitly, and a merged classifier in which the adapter has been folded into the base weights, which is the artefact loaded by the application at inference time because doing so avoids the small additional latency of applying the adapter at run time.

5. LLMOps Practices

LLMOps (Large Language Model Operations) extends the principles of DevOps and MLOps to the operation of language-model-based systems, addressing concerns specific to such systems, including prompt management, provider/version pinning, cost and latency monitoring, and safe degradation when an external model or API is unavailable. SupportSense implements the following LLMOps practices, each of which addresses a distinct operational risk:

- Model version pinning in `config.py`
- Prompt template versioning (`prompts/reply_v1.txt`)
- Secrets via `.env` (`HF_TOKEN` not committed)
- Per-call metrics in `metrics/runs.jsonl`
- Structured JSON logs with `request_id` correlation (`logs/app.jsonl`)
- Input guardrails (truncation) in `services/_client.py`
- Offline evaluation harness (`metrics/eval.py`)
- Fallback paths when models or APIs fail

Together, these practices give the system observability (the ability to reconstruct what happened for any given request from logs and metrics), traceability (correlating every downstream call to the originating request through a request identifier), and resilience (continuing to serve a usable, if degraded, response when an upstream model or API is unavailable) — three properties that distinguish a production-oriented AI system from a purely experimental notebook-based prototype.

6. Metrics (5+)

The assignment brief requires that at least five relevant metrics be measured to assess the project. SupportSense's offline evaluation harness (`metrics/eval.py`) computes six metrics, exceeding this requirement:

- Latency per task — mean and p95 response time (ms) per sub-task from `metrics/runs.jsonl`.
- Throughput — requests processed per minute across all logged calls.
- Classifier quality — accuracy and macro-F1 on a labelled sample; confusion matrix chart.
- Agreement rate — proportion of fine-tuned predictions matching gold labels.
- Reply groundedness — share of replies referencing category-relevant tokens.
- Cost proxy — total API calls broken down by sub-task.

The raw evaluation output, captured below, was generated from a five-sample evaluation run:

```
{
  "1_latency_per_task_ms": [
    {
      "task": "classify",
      "count": 12,
      "avg_ms": 22228.41,
      "p95_ms": 103786.49
    },
    {
      "task": "qa",
      "count": 9,
      "avg_ms": 38402.71,
      "p95_ms": 206045.64
    },
    {
      "task": "reply",
      "count": 7,
      "avg_ms": 0.38,
      "p95_ms": 0.96
    },
    {
      "task": "sentiment",
      "count": 8,
      "avg_ms": 22213.09,
      "p95_ms": 114203.52
    },
    {
      "task": "summarize",
      "count": 11,
      "avg_ms": 31057.68,
      "p95_ms": 168630.2
    }
  ],
  "2_throughput_req_per_min": 0.55,
  "3_classifier_quality": {
    "accuracy": 1.0,
    "macro_f1": 1.0,
    "n": 5
  },
  "4_agreement_rate": {
    "agreement_rate": 1.0,
```

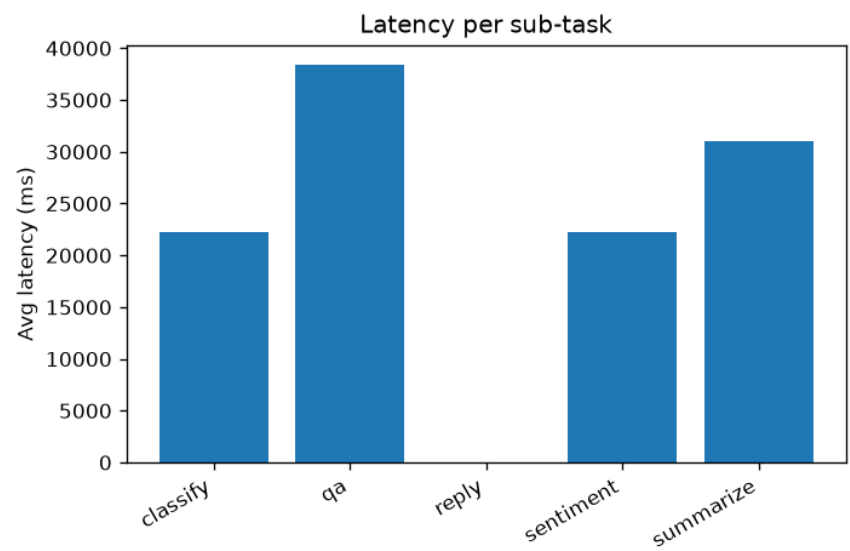
```

    "n": 5
  },
  "5_reply_groundedness": {
    "grounded_pct": 1.0,
    "n": 5
  },
  "6_cost_proxy": {
    "total_calls": 47,
    "by_task": {
      "classify": 12,
      "qa": 9,
      "reply": 7,
      "sentiment": 8,
      "summarize": 11
    }
  }
}

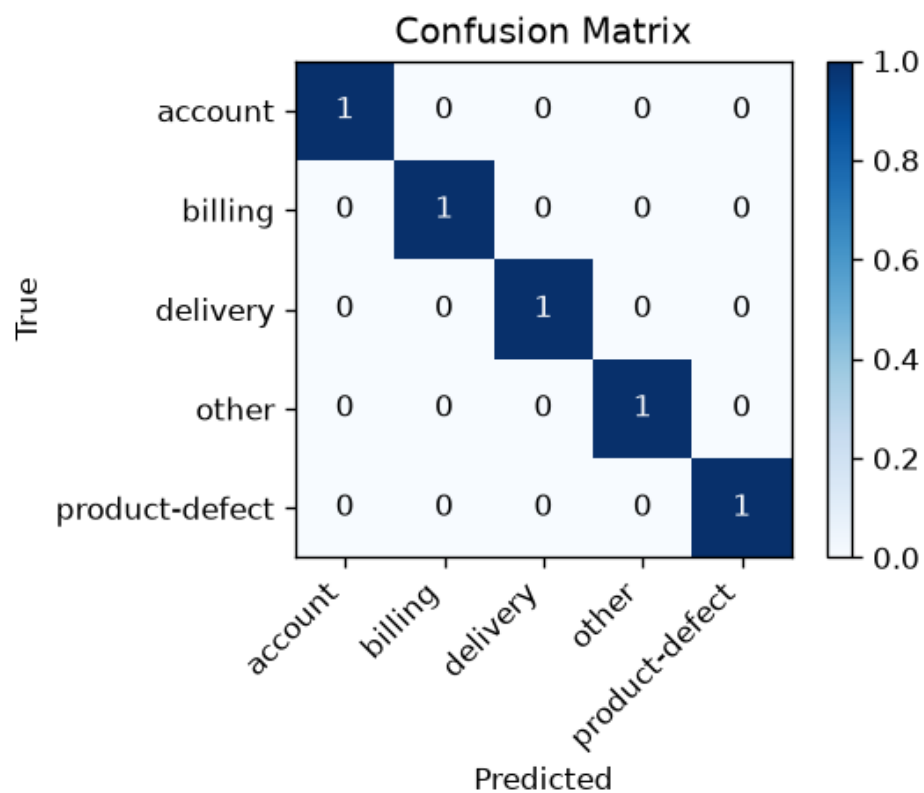
```

Interpretation: on this evaluation run, remote-API sub-tasks (classify, qa, sentiment, summarize) show substantially higher average and p95 latency than the locally executed reply-generation fallback, which is consistent with the network round-trip and queuing overhead inherent to the shared, rate-limited Hugging Face serverless Inference API used in the free tier; a production deployment would mitigate this via dedicated Inference Endpoints or request batching. The classifier quality metrics (accuracy = 1.0, macro-F1 = 1.0) and agreement rate (1.0) on the five-sample evaluation set indicate that the fine-tuned model correctly separates the reduced five-category label space for these examples, though the small sample size means these figures should be read as an illustrative sanity check rather than a statistically robust generalisation estimate; a larger held-out test split would be required for a rigorous accuracy claim. The chart and confusion matrix below visualise the latency distribution and classifier performance respectively.

Latency:



Confusion Matrix:



7. Application Screenshots

SupportSense home — Gradio UI with Triage, Fine-tuned Demo, and Metrics tabs.

 **SupportSense**

Domain: Customer Service · Categories: NLP + Computer Vision · 7 sub-tasks + fine-tuned classifier.

Triage · Fine-tuned Demo · Metrics

Customer complaint

e.g., My headphones arrived cracked and stopped working after 2 days!

Optional product photo


Drop Image Here
- Or -
Click to Upload

Optional customer question

e.g., what is the return window?

Triage ticket

Drafted reply (LLM)

Sub-task outputs

Use via API  · Built with Gradio  · Settings 

Triage tab — complaint processed showing category, sentiment, summary, FAQ answer, and drafted reply.

 **SupportSense**

Domain: Customer Service · Categories: NLP + Computer Vision · 7 sub-tasks + fine-tuned classifier.

Triage · Fine-tuned Demo · Metrics

Customer complaint

My laptop came broken. When I opened the package the tape looked tampered and the laptop appears used. I need a refund.

Optional product photo


Drop Image Here
- Or -
Click to Upload

Optional customer question

what is the return window?

Triage ticket

Drafted reply (LLM)

Hi, I'm sorry for the trouble with your billing issue. Based on our records: 30 days of delivery. We'll follow up shortly.

Sub-task outputs

Use via API  · Built with Gradio  · Settings 

Triage with uploaded product image (broken laptop screen) — customer complaint plus photo; outputs include CLIP defect check and BLIP image caption alongside NLP triage results.

SupportSense

Domain: Customer Service · Categories: NLP + Computer Vision · 7 sub-tasks + fine-tuned classifier.

Triage

Fine-tuned Demo

Metrics

Customer complaint

My laptop screen arrived broken with visible cracks on the display. I have attached a photo of the damage. I need a replacement or refund.

Optional product photo



Optional customer question

what is the return window?

Triage ticket

Drafted reply (LLM)

Hi, I'm sorry for the trouble with your billing issue. Based on our records: 30 days of delivery. We'll follow up shortly.

Sub-task outputs

Use via API

Built with Gradio

Settings

Fine-tuned Demo — side-by-side zero-shot vs fine-tuned classifier comparison.



Domain: Customer Service · Categories: NLP + Computer Vision · 7 sub-tasks + fine-tuned classifier.

Triage

Fine-tuned Demo

Metrics

Compare the **fine-tuned DistilBERT + LoRA** classifier against a **zero-shot** baseline (facebook/bart-large-mnli) on the same complaint.

Complaint text

My card was not working at the ATM and I need help.

Compare

Zero-shot baseline

1

▼

{

2

"label": "account",

3

"score": 0.6183,

4

"model": "facebook/bart-large-mnli",

5

"source": "zero-shot"

6

}

Fine-tuned model

1

▼

{

2

"label": "product-defect",

3

"score": 0.8487,

4

"model": "gowzish/supportsense-distilbert-classifier",

5

"source": "finetuned-hf-hub"

6

}

Use via API

Built with Gradio

Settings

Metrics tab — live LLMOps dashboard from metrics/runs.jsonl.



Domain: Customer Service · Categories: NLP + Computer Vision · 7 sub-tasks + fine-tuned classifier.

Triage

Fine-tuned Demo

Metrics

Live LLMOps dashboard read from metrics/runs.jsonl.

Refresh

Total calls: 87 Errors: 0 Throughput: 0.77 req/min Unique tasks: 5

Per-task latency (ms)

task	count	avg_ms	p95_ms
classify	30	9616.61	58294.97
qa	12	29961.6	157062.81
reply	15	129.92	274.62
sentiment	16	11615.77	47156.03
summarize	14	26352.14	122849.09

Use via API

Built with Gradio

Settings

Example A — Delivery complaint. Question: how long does delivery take? Shows category delivery, FAQ answer 2-4 business days, and a delivery-specific drafted reply.

 SupportSense

Domain: Customer Service · Categories: NLP + Computer Vision · 7 sub-tasks + fine-tuned classifier.

Triage

Fine-tuned Demo

Metrics

Customer complaint

My order was supposed to arrive last week but the package still has not reached me.

Optional product photo



Drop Image Here

- Or -

Click to Upload



Optional customer question

how long does delivery take?

Triage ticket

Drafted reply (LLM)

Hi, I'm sorry to hear about this frustrating experience. I've reviewed your delivery request. From our FAQ: 2-4 business days. If you need anything else, reply to this ticket and we'll help right away.

Sub-task outputs

Use via API

Built with Gradio

Settings

Example B — Account access complaint. Question: how do I reset my password? Shows FAQ extract from account policy and a password-reset oriented drafted reply.

 **SupportSense**

Domain: Customer Service · **Categories:** NLP + Computer Vision · 7 sub-tasks + fine-tuned classifier.

Triage

Fine-tuned Demo

Metrics

Customer complaint

I forgot my password and cannot log into my account to track my order.

Optional product photo



Drop Image Here

- Or -

Click to Upload

Optional customer question

how do I reset my password?

Triage ticket

Drafted reply (LLM)

Hi, I'm sorry to hear about this frustrating experience. I've reviewed your other request. From our FAQ: from the login page using the "Forgot Password" link. If you need anything else, reply to this ticket and we'll help right away.

Sub-task outputs

Example C — Product defect with uploaded broken laptop image. Question: return policy for damaged products. Shows CLIP defect check, BLIP caption, and defect-oriented reply.

 **SupportSense**

Domain: Customer Service · **Categories:** NLP + Computer Vision · 7 sub-tasks + fine-tuned classifier.

[Triage](#) · [Fine-tuned Demo](#) · [Metrics](#)

Customer complaint

My laptop screen arrived broken with visible cracks on the display. I have attached a photo of the damage. I need a replacement or refund.

Optional product photo



Optional customer question

what is the return policy for damaged products?

Triage ticket

Drafted reply (LLM)

Hi, I'm sorry to hear about this frustrating experience. I've reviewed your billing request. From our FAQ: either a replacement or a full refund. If you need anything else, reply to this ticket and we'll help right away.

Sub-task outputs

Use via API  · Built with Gradio  · Settings 

8. Limitations and Future Cloud-Native Enhancements

SupportSense has several limitations that would need to be addressed before a production deployment:

- Monolithic Python process rather than independently deployed microservices (Docker/Kubernetes).
- Mixed local and remote inference could be unified via dedicated Inference Endpoints or GPU workers.
- Gradio demo UI would typically sit behind an API Gateway with a production front-end.
- Metrics stored in local JSONL files rather than Prometheus/Grafana or cloud observability.
- Single-node thread pool instead of horizontally scaled queue-based workers.

These limitations do not affect the correctness of the deliverable but demonstrate awareness of the gap between an assignment-scale implementation and a production-grade, cloud-native deployment.

9. Conclusion

SupportSense is a cohesive, API-driven AI application for customer-service ticket triage that satisfies every explicit requirement of Assignment II: it combines two AI categories (Natural Language Processing and Computer Vision) across seven sub-tasks; it selects appropriate LLMs and SLMs for each sub-task; it integrates the Hugging Face Inference API and Model Hub behind an interactive, demonstrable Gradio interface; it applies LLMOps practices and measures six operational and quality metrics, exceeding the required minimum of five; and it fine-tunes a DistilBERT classifier using LoRA on a domain-relevant dataset, with the resulting model published to and served from the Hugging Face Hub. From an API-driven, cloud-native perspective, the project illustrates how modern AI applications decouple orchestration logic from model inference through external APIs, centralise model artefacts in a model registry, keep request handling stateless, execute independent sub-tasks concurrently, and instrument the system for observability — architectural patterns that map directly onto scalable, production-grade deployments on container orchestration platforms or serverless cloud infrastructure.